

TUNIX SOFTWARE AND EMBEDDED SYSTEMS

**CORE MODULES: MEMORY, ERROR, & SELF-TEST
FORKS**

VERSION: 3.1

DOCUMENT REVISION: 2.0 | ARCHITECTURE: AVR / ATMEGA328P

1. Introduction

The Tunix Core Modules v3.1 aim to achieve a clean, type-safe, optimized, ready-to-use, and easy infrastructure management framework on Arduino Boards (ATMEGA328P) and other AVR-based microcontrollers. This version introduces modern C++ enhancements such as strongly-typed error enumerations (`enum class`), version structure operator overloading for robust firmware validation, modular namespace isolation, and direct hardware register SRAM integrity testing without heavily consuming Flash memory or dynamic RAM.

2. Associated Files

The Tunix Core requires specific file pairs to be added to the root project folder alongside the main `*.ino` file. Modules operate independently with minimal dependencies, communicating primarily through shared constants and typed error codes.

- **Memory Fork:** `tunixMemoryManager.h`, `tunixMemoryManager.cpp`
- **Error Fork:** `tunixErrorManager.h`, `tunixErrorManager.cpp`
- **Self-Test Fork:** `tunixSelfTest.h`, `tunixSelfTest.cpp`
- **Constants:** `constants.h`

3. Module Specifications & Protocols

3.1. Memory Manager Fork

Provides automated EEPROM address layout management, structured settings serialization, and firmware validation logic. Memory allocation begins with the fixed `VersionInfo` struct address at `0x00`, automatically calculating user settings address offset via `sizeof()`.

- **Firmware Validation:** Compares runtime firmware version against saved EEPROM version using `VersionInfo` struct operator overloading (`<` and `==`). Handles upgrade and downgrade states safely.
- **Namespace & Struct Integration:** Integrates with `SystemVersion` namespace (`FIRMWARE`, `MIN_SUPPORTED`) and seamlessly handles custom setting structures without manual byte-shifting.

- **Factory Reset Protocol:** Restores default parameters (defaultSettings) to EEPROM, updates saved VersionInfo, and provides delay buffer before system hold/reset.

3.2. Error Manager Fork

Acts as a centralized safety dispatch registry for runtime exceptions. It intercepts strongly-typed ErrorCode values triggered across modules, converts them to numerical identifiers, and executes predefined safety protocols.

3.3. Self-Test Fork

Executes boot-level hardware integrity checks. This version does not include changes to Self-Test Fork.

- **RAM Integrity Test:** Scans the dynamic SRAM area using 0x5555 and 0xAAAA bitwise patterns to detect faulty registers. Restores original memory states post-test to prevent data corruption.
- **Free RAM Calculation:** Returns available SRAM in bytes via pointer arithmetic (__heap_start and __brkval).

3.3. Constants

Centralizes hardware pin definitions (Pins namespace), versioning structures (VersionInfo), system version declarations (SystemVersion namespace), and enum class ErrorCode definitions.

4. Standardized Error Codes Reference (Quick Look-up)

Use this reference table during runtime debugging and system auditing.

Error Code	Definition	Module Origin	Recommended Protocol
00	SYSTEM_OK	System	Continue Boot
11	FIRMWARE_VERSION_ERR OR	Memory Manager	Force Factory Reset

Error Code	Definition	Module Origin	Recommended Protocol
12	VERSION_NOT_SUPPORTED	Memory Manager	Force Factory Reset
13	DOWNGRADED_FIRMWARE	Memory Manager	Recommend Factory Reset
14	PROGRAM_LOOP_FAILURE	Main Loop	Reset Device
15	RAM_FAILURE	Self-Test	Halt System (Critical)

5. Compatibility & Limitations

- Tunix Memory Manager integrates VersionInfo structs. Overwriting memory without updating versioning triggers validation protection.
- Self-Test SRAM bitwise pattern scanning relies on AVR ATMEGA328P heap/stack memory pointers (__heap_start, __brkval, RAMEND). Non-AVR architectures (ESP32, STM32) require architecture conditional blocks (#if defined(__AVR__)).
- Maximum user settings structure size is constrained by the microcontroller's physical EEPROM boundary minus sizeof(VersionInfo) (3 Bytes).

6. EEPROM Memory Map (Address Layout)

The memory map below details the default ATMEGA328P EEPROM layout. Addresses are automatically calculated via compile-time sizeof() evaluation.

Address Range	Data Structure	Size (Bytes)	Description & Variables
0x00 - 0x02	versionStruct	3 Bytes	Firmware Version Data (Major, Minor, Patch).
0x03 - 0x??	settingsStruct	Dynamic	User-defined settings. Starts immediately after version struct.
0x?? - EOF	Unallocated	Variable	Available free space for future struct additions or raw data arrays.

Warning: Do not manually overwrite addresses 0x00 to 0x02 using raw EEPROM.write() or EEPROM.update(). This will corrupt the VersionInfo structure and force ErrorCode::FIRMWARE_VERSION_ERROR or ErrorCode::DOWNGRADED_FIRMWARE during boot validation.

7. Functions and Classes

```
// --- Constants.h ---
struct VersionInfo {
    byte major;
    byte minor;
    byte patch;
    bool operator<(const VersionInfo& other) const;
    bool operator==(const VersionInfo& other) const;
};

enum class ErrorCode : byte {
    NONE = 0, FIRMWARE_VERSION_ERROR = 11, VERSION_NOT_SUPPORTED = 12,
    DOWNGRADED_FIRMWARE = 13, PROGRAM_LOOP_FAILURE = 14, RAM_FAILURE = 15
};
```

```

// --- TunixMemoryManager.h ---
class TunixMemoryManager {
private:
    int _EEPROMSize;
public:
    struct settingsStruct { };
    settingsStruct settings;
    settingsStruct defaultSettings;
    VersionInfo eepromVersion;
    TunixMemoryManager(int EEPROMSize);
    void getBasicMemory();
    void saveBasicMemory();
    void factoryReset();
    ErrorCode firmwareValidate();
};

// --- TunixErrorManager.h ---
class TunixErrorManager {
public:
    TunixErrorManager();
    void errorHandler(ErrorCode code);
};

// --- TunixSelfTest.h ---
class TunixSelfTest {
private:
    bool runRAMTest();
public:
    TunixSelfTest();
    int getFreeRAM();
    ErrorCode selfTest();
};

```

8. Example

8.1. Initializing the modules:

```

// Main setup initialization
TunixMemoryManager memory(1024); // 1024 Bytes EEPROM size for ATmega328P
TunixErrorManager errorManager;
TunixSelfTest deviceTest;

```

8.2. System Boot Squence & Execution Flow

```
void setup(){
    // 1. Hardware Self-Test
    ErrorCode testResult = deviceTest.selfTest();
    if (testResult != ErrorCode::NONE) {
        errorManager.errorHandler(testResult); // Intercept RAM failure
    }

    // 2. Load EEPROM & Validate Firmware
    memory.getBasicMemory();
    ErrorCode versionResult = memory.firmwareValidate();
    if (versionResult != ErrorCode::NONE) {
        errorManager.errorHandler(versionResult);
    }

    // 3. Normal Operation
    memory.saveBasicMemory();
}
```